```
  <ul>
»   <% cache @products do %>
      <% @products.each do |product| %>
»       <% cache product do %>
          <li class='flex mb-6'>
            <%= image_tag(product.image_url,
              class: 'object-contain w-40 h-48 shadow mr-6') %>

            <div>
              <h2 class="font-bold text-lg mb-3"><%= product.title %></h2>

              <p>
                <%= sanitize(product.description) %>
              </p>

              <div class="mt-3">
                <%= number_to_currency(product.price) %>
              </div>
            </div>
          </li>
»       <% end %>
      <% end %>
»   <% end %>
  </ul>
  </div>
```

In addition to bracketing the sections, we identify the data to associate with each: the complete set of products for the overall store and the individual product we're rendering with the entry. Whenever the specified data changes, the section will be rerendered.

Bracketed sections can be nested to arbitrary depth, which is why those in the Rails community have come to refer to this as "Russian doll" caching.[45]

With this, we're done! Rails takes care of all of the rest, including managing the storage and deciding when to invalidate old entries. If you're interested, you can turn all sorts of knobs and make choices as to which backing store to use for the cache. It's nothing you need to worry about now, but it might be worth bookmarking the overview page of Caching with Rails in the Ruby on Rails Guides.[46]